Planning as Autoregressive Sequence Modelling: Decoding Strategies

Dian Kruger
Shane Josias

Abstract

Planning concerns designing an agent living within an environment to find a sequence of actions that transforms the environment from an initial state to a desired state. Recently, planning tasks have been modelled as a sequence prediction problem, where a supervised learning model is trained to predict sequences of actions or states. This is attractive because flexible models, such as Transformers, can be used for such tasks. This work focuses on decoding: how to generate trajectories from a trained sequence model in order to solve a discrete maze navigation problem, considering both fully observed scenarios, where the complete maze layout is available to the planner, and partially observed scenarios, where the model can only see a local neighbourhood and a goal displacement vector at each position in the maze. This work compares greedy decoding and beam search as decoding strategies. Greedy decoding always selects the most probable next action at every time step, while beam search considers multiple sequences of tokens, which could be actions or observations, as candidates and compares these sequences as opposed to individual tokens. Decoding strategies were compared in terms of completion rate, average steps until termination, average computation time per trajectory, and average computation time per step. Beam search consistently required more computation time per step compared to greedy decoding in the experiments that were conducted, and in the fully observed maze environment investigated, beam search also achieved a higher completion rate and lower average steps until termination than greedy decoding. In the partially observed setting, however, the relative performance of the strategies in terms of completion rate and average steps until termination differed based on the training data used to fit the model.

1 Introduction

Planning is a central problem in artificial intelligence. Given an environment, and an agent that can interact with it, planning is concerned with finding a sequence of actions that moves the environment from an initial state to a desired goal state [11]. In this project, the planning problem is studied in a discrete 2D maze environment, where the task is to generate a valid trajectory from a given start position to a specified goal position.

One way to approach such a problem is to treat trajectories as data and learn a probability distribution over successful trajectories from examples. From this perspective, planning

can be viewed as inference in a learned generative model [3]. If the model is trained on valid and efficient demonstration trajectories, then complete plans can be generated autoregressively by repeatedly predicting the next token in a trajectory sequence.

The decoding rule used during this generation process is important. At each step, the learned model may assign non-negligible probability to multiple possible continuations, and different decoding strategies can therefore produce trajectories with different levels of validity, efficiency, and computational cost [3, 9]. In the setting considered in this work, decoding is based on model probability rather than on a reward-maximising objective, so the planning problem is approached through imitation learning rather than offline reinforcement learning [3].

This report investigates how the choice of decoding strategy affects planning performance under this formulation. A Transformer-based trajectory model is trained through next-token prediction on a dataset of valid maze trajectories, and the resulting model is evaluated using greedy decoding and beam search with different beam widths. The aim is to determine how these decoding rules affect goal-reaching success, steps until termination, and computational cost.

The comparison is conducted in both fully observed and partially observed settings. In the fully observed setting, the model is provided with sufficient information to determine the complete states of the environment at every timestep. In the partially observed setting, the model instead receives only a local neighbourhood around its current position together with the relative displacement of the goal, requiring it to use trajectory history to reason about unobserved aspects of the environment. The models used in these comparisons are trained on optimal trajectories, while an additional experiment in the partially observed setting investigates how decoding performance changes when the model is trained on a larger dataset containing trajectories of varying degrees of optimality.

The results suggest that broader searches, which retain and compare more candidate trajectories before selecting an action, are most beneficial when candidate future trajectories can be evaluated using sufficiently accurate information about the states or observations that follow hypothetical actions. In the fully observed setting, where these states can be determined exactly, beam search substantially outperforms greedy decoding. Under partial observability, however, this advantage seems to depend on how accurately future observations can be predicted by the model.

2 Background

2.1 Generative Models

Generative models are types of machine learning systems that aim to represent a probability distribution over data, or to draw samples from that distribution [10]. Specifically, if $\mathbf{x}$ is a collection of features observed from an object or event, like words in a sentence or coordinates in a maze, and $p(\mathbf{x})$ is the true joint distribution of those features, a generative model learns an approximation $\hat{p}(\mathbf{x})$ of this distribution from training observations [10]. Once such a distribution has been learned, it can be used to generate new observations

that are similar to the observations in the training data by sampling from $\hat{p}(\mathbf{x})$ [10]. In practice, assumptions are often made to simplify the learning problem, such as assuming a particular parametric form for the distribution (e.g., Gaussian), so instead of learning an arbitrary distribution, the model learns a specific family of distributions parameterised by $\boldsymbol{\theta}$ [10].

If the data's true distribution $p(\mathbf{x})$ is assumed to belong to a family of distributions $\{p_{\boldsymbol{\theta}}(\mathbf{x})\}$, the model must learn the parameter value $\hat{\boldsymbol{\theta}}$ such that $p_{\hat{\boldsymbol{\theta}}}(\mathbf{x})$ is a good approximation of $p(\mathbf{x})$. One way to formalise this is through the Kullback–Leibler (KL) divergence, which measures how one probability distribution diverges from another [10]. It is defined as

$$D_{\mathrm{KL}}(p(\mathbf{x})\|p_{\boldsymbol{\theta}}(\mathbf{x})) = \mathbb{E}_{\mathbf{x}\sim p(\mathbf{x})}\left[\log \frac{p(\mathbf{x})}{p_{\boldsymbol{\theta}}(\mathbf{x})}\right]. \tag{1}$$

In this setting, learning can be viewed as choosing $\boldsymbol{\theta}$ to minimise $D_{\mathrm{KL}}(p(\mathbf{x})\|p_{\boldsymbol{\theta}}(\mathbf{x}))$. Expanding this expression gives

$$D_{\mathrm{KL}}(p(\mathbf{x})\|p_{\boldsymbol{\theta}}(\mathbf{x})) = \mathbb{E}_{\mathbf{x}\sim p(\mathbf{x})}[\log p(\mathbf{x})] - \mathbb{E}_{\mathbf{x}\sim p(\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x})]. \tag{2}$$

The first term does not depend on $\boldsymbol{\theta}$, so minimising the KL divergence is equivalent to maximising $\mathbb{E}_{\mathbf{x}\sim p(\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x})]$, that is, the expected log-likelihood of the data under the model [10]. The process of finding $\hat{\boldsymbol{\theta}}$ that maximises the likelihood of the training data under $p_{\boldsymbol{\theta}}(\mathbf{x})$ is known as maximum likelihood estimation (MLE), and is a common approach to training generative models [10].

2.2 The Planning Problem

Broadly, automated planning is the field of research addressing the problem of finding an ordered or partially ordered series of actions that changes an environment from its initial state to a desired goal state [11]. One can imagine an agent living in an environment, where the agent can take actions that change the state of the environment. Planning is then concerned with determining a sequence of actions to be executed by the agent to achieve some goal.

In this work, the planning problem is to generate a valid trajectory from a given starting position to a specified goal position in a 2D maze environment, as shown in Figure 1. In this environment, a trajectory consists of a sequence of maze positions from the start location to the goal location, together with the actions connecting consecutive positions, such that the path does not pass through walls. Assuming a discrete grid-based representation, maze coordinates are integers identifying grid cells. A trajectory can therefore be represented as a sequence of coordinates in which each consecutive pair corresponds to adjacent non-wall cells, with adjacency restricted to horizontal and vertical moves.

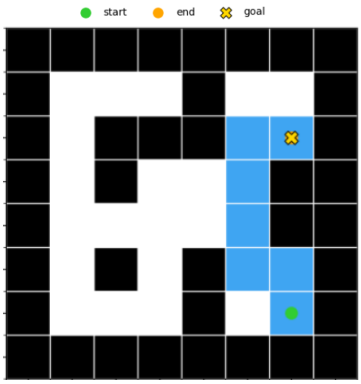


Figure 1: Example of a valid trajectory in a discrete 2D maze environment.

2.3 Approaches to Planning

Reinforcement learning (RL) is a machine learning paradigm in which a system learns to choose appropriate sequences of actions by interacting with an environment in order to maximise cumulative reward, making it a natural fit for planning problems, since the reward can be defined in terms of reaching a goal state efficiently [1]. Offline reinforcement learning is a variant of RL in which the policy that determines the agent's actions is learned from a fixed dataset of previously collected trajectories, rather than through repeated interaction with the environment [3]. Imitation learning is a related paradigm in which a model learns to reproduce the behaviour demonstrated in a dataset. Similarly to offline RL, imitation learning can be applied to a dataset of trajectories, but the objective is to generate trajectories that are similar to those in the dataset, as opposed to maximising cumulative reward as in offline RL [3]. Similarly to RL, imitation learning can solve a planning problem, but only if the dataset contains trajectories that are valid and efficient for the planning task at hand [3].

In reinforcement learning, the term "planning" often refers to using a model of the environment to consider the likely consequences of possible actions before deciding which action to take. An important distinction must thus be made between planning as a problem of generating a sequence of actions to reach a goal, and planning as a process of using a model to improve decision-making in reinforcement learning. In this work, the former interpretation is used.

2.4 Generative Modelling for Planning

If a dataset of valid trajectories is available, a generative model can be trained to learn the distribution over these trajectories. Once trained, the model can be used to generate new trajectories by sampling from the learned distribution. This approach treats planning as a problem of inference in a probabilistic model [3]. So if we define a trajectory $\boldsymbol{\tau}$ with T action steps as:

$$\boldsymbol{\tau} = (s_1, a_1, s_2, a_2, \ldots, s_T, a_T), \tag{3}$$

where s_t is the state (coordinates) at time step t and a_t is the action (moving up, down, left or right) taken at time step t, the trained generative model defines an approximation $p_{\hat{\boldsymbol{\theta}}}(\boldsymbol{\tau})$ to the distribution over valid trajectories. Instead of directly modelling $p_{\hat{\boldsymbol{\theta}}}(\boldsymbol{\tau})$, an autoregressive model factorises this distribution into a product of conditional probabilities:

$$p_{\hat{\boldsymbol{\theta}}}(\boldsymbol{\tau}) = \prod_{k=1}^{2T} p_{\hat{\boldsymbol{\theta}}}(z_k \mid z_{<k}), \tag{4}$$

where z_k denotes the kth token in the flattened trajectory sequence. Specifically, $z_{2t-1} = s_t$ and $z_{2t} = a_t$ for $t = 1, \ldots, T$, so odd-indexed tokens are states and even-indexed tokens are actions. The trajectory prefix $z_{<k}$ denotes the sequence of preceding tokens up to z_{k-1} [10]. This simplifies the supervised learning problem to estimating $p_{\boldsymbol{\theta}}(z_k \mid z_{<k})$, so given a prefix of trajectory tokens, predict the probability of the next token in the sequence.

A complete trajectory is then generated autoregressively by repeatedly predicting the next token until a full plan is produced, with the decoding strategy determining how each next-token choice is made. The choice of decoding strategy is important because the model's predicted distribution may assign non-negligible probability mass to multiple valid continuations at each step, and the continuation selected during decoding can significantly affect both the quality of the final trajectory and the computational cost of generation [3]. Since the learned distribution is only an approximation to the true trajectory distribution, it may also assign non-negligible probability to invalid continuations. As a result, the decoding strategy can influence whether a valid trajectory is generated at all. Even when multiple valid trajectories are possible, some may be more desirable than others, for example because they require fewer steps, and the decoding strategy can therefore affect whether a more efficient trajectory is produced [3, 9].

2.5 Decoding Methods

Direct sampling from $p_{\hat{\boldsymbol{\theta}}}(\boldsymbol{\tau})$ is challenging, since the number of possible trajectories grows exponentially with trajectory length, so explicitly calculating and storing the probabilities for all complete trajectories quickly becomes computationally infeasible [10]. This creates a need for decoding algorithms to approximately sample from, or search within $p_{\hat{\boldsymbol{\theta}}}(\boldsymbol{\tau})$.

2.5.1 Greedy Decoding

Greedy decoding is the simplest method for generating sequences from $p_{\hat{\boldsymbol{\theta}}}(\boldsymbol{\tau})$ [9]. At each step t during the autoregressive generation of a trajectory, the algorithm selects the next token $\hat{z}_t$ as follows:

$$\hat{z}_t = \arg\max_{z \in V} p_{\hat{\boldsymbol{\theta}}}(z \mid z_{<t}) \tag{5}$$

where V is the set of all possible tokens, and $z_{<t}$ denotes the values obtained in the previous steps of greedy decoding. Greedy decoding thus always selects the locally optimal choice for the next token in the sequence, meaning it chooses the token with the highest conditional probability given the current prefix [9]. Greedy decoding is therefore deterministic and computationally efficient, but does not necessarily find the globally most probable trajectory, which is the complete trajectory with the highest probability under $p_{\hat{\boldsymbol{\theta}}}(\boldsymbol{\tau})$ [9].

2.5.2 Beam Search

Beam search extends greedy decoding by retaining multiple candidate sequences at each step rather than only the single most likely continuation [9]. The number of retained candidates is fixed in advance and is known as the beam width B. At each stage, all valid one-step continuations of the current candidates are scored autoregressively under the model, and the top B highest-scoring sequences are kept. If a retained candidate reaches the goal, the highest-scoring goal-reaching candidate is returned. Otherwise, search stops at the action limit and returns the highest-scoring retained trajectory.

The beam search algorithm for the fully observed experiment is summarised in Listing 1. Its partially observed adaptation is described separately in Section 6.2.3.

Beam Search [3]

Require maximum number of actions H, beam width B, maze layout, start state s_{start}, and goal state s_{goal}
Initialize $\mathcal{T}_0$ with the prefix containing the maze identifier, start and goal tokens, and initial state token; set its score to zero
for $t = 0, \ldots, H-1$ **do**
    Keep any goal-reaching trajectories in $\mathcal{T}_t$ unchanged as candidates
    For each other $\boldsymbol{\tau} \in \mathcal{T}_t$ and each valid action a from its final state, append a and the resulting state s' to form a candidate $\boldsymbol{\tau}'$
    Set $S(\boldsymbol{\tau}') \leftarrow S(\boldsymbol{\tau}) + \log p_{\boldsymbol{\theta}}(a \mid \boldsymbol{\tau})$
    $\mathcal{T}_{t+1} \leftarrow$ the B highest-scoring candidates
    if any trajectory in $\mathcal{T}_{t+1}$ reaches s_{goal}, **return** the highest-scoring goal-reaching trajectory
end for
Return the highest-scoring trajectory in $\mathcal{T}_H$ if the action limit is reached

Listing 1: Pseudocode for beam search in the fully observed setting.

Here, t denotes the current decoding step, $\mathcal{T}_t$ is the set of retained partial trajectories at that step, and $S(\boldsymbol{\tau})$ is a trajectory's accumulated score. The resulting state s' is calculated from the current state and selected action. The beam width bounds the number of retained trajectories, while the number of candidates depends on how many valid actions extend them. Maintaining the candidate scores remains computationally feasible when the beam width is sufficiently small [9].

Larger beam widths allow the algorithm to consider a greater number of candidate trajectories. As a result, the trajectory returned by beam search will typically be at least as highly scored under the model as the trajectory returned with a smaller beam width, with a beam width of one corresponding to greedy decoding. This broader search, however, requires the model scores of more candidate trajectories to be computed and maintained, so increasing the beam width also increases the computational cost of decoding [9].

Both greedy decoding and beam search, as defined above, aim to find trajectories with high probability under the learned model, and therefore correspond naturally to an imitation learning setting. If, however, the decoding objective is defined in terms of predicted reward or return rather than model probability alone, then the same decoding procedures can instead be used in an offline reinforcement learning setting.

2.6 Transformer Based Generative Models

The Transformer is a type of neural network that has become the standard for sequence modelling tasks, notably in natural language processing [9]. Even though the original Transformer was designed for language, the fact that it models sequences of tokens makes it applicable to our trajectory modelling problem, where the tokens could be states and/or actions in a trajectory [3].

The key components of the Transformer are now briefly introduced. For intuition, the discussion refers primarily to natural language processing examples, where the role of these components is often easier to interpret than in trajectory modelling.

2.6.1 Input Representation

Before raw data enters the Transformer, it must be converted into a high dimensional vector representation as follows.

Tokenisation. The input sequence is broken down into discrete tokens. In an NLP (Natural Language Processing) scenario, this could involve splitting text into words, subwords or characters [9]. For the running example in language processing, it is assumed that the input has been tokenised at the word level.

Embeddings (E). Each token ID is used to index into an embedding matrix E, which maps the discrete ID to a dense vector of dimensionality d. This matrix has shape $|V| \times d$, where $|V|$ is the vocabulary size, i.e. the total number of distinct tokens that the model can represent [9]. When tokens correspond to words, the embedding can be interpreted as capturing semantic and syntactic properties of those words. Intuitively, words with similar meanings, such as "big" and "large," may be represented by vectors that are close to one another in the embedding space [9].

Positional Encoding. Since the Transformer processes sequences in parallel, it does not inherently encode the order of tokens, so a positional encoding is added to the token embeddings to ensure that the position of each token in the sequence is represented. This is done by adding a positional encoding vector to each token embedding [9].

Input Formation. For a sequence of N tokens, the individual d-dimensional vector embeddings are placed into a single matrix X of size $N \times d$. This matrix serves as the input for the model [9].

2.6.2 Transformer Blocks

Each block consists of a multi-head self-attention mechanism followed by a feedforward neural network, with each sublayer followed by a layer normalisation and a residual connection [9]. A useful mental model is that each time the input passes through a sublayer, the model updates the token representations so that they better capture the information relevant to the task at hand. For example, in the NLP context, the model may refine the representation of a word so that it better reflects its meaning in the context of the sentence [9]. Each block is described as follows.

Layer Normalisation. For training stability, layer normalisation is applied to ensure the values in the sublayers remain within a reasonable range. For a vector x, it is calculated as:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sigma} + \beta \tag{6}$$

where μ is the mean, σ the standard deviation of the vector's elements, and γ and β are learned parameters that scale and shift the normalised output [9]. $\odot$ denotes an element-wise vector product.

Multi-Head Self-Attention. This is the core mechanism that allows the model to incorporate information from all tokens in the sequence when processing each token [9]. Under the interpretation that each sublayer progressively refines word representations, the self-attention mechanism allows the representation of each word to be updated in light of the representations of all other words in the sequence. For example, if the word "fly" is preceded by the word "the", the model may adjust its representation in a way that reflects the nominal sense of "fly" (the insect) rather than the verbal sense, which is the action of flying. Each input vector x_i is mapped into three learned representations: a Query (Q), a Key (K), and a Value (V). For an input matrix X, these are obtained by learned linear projections:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V \tag{7}$$

where W_Q, W_K, and W_V are learned weight matrices [9].

The attention for a single head is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{8}$$

The scaling factor $\sqrt{d_k}$ prevents the dot products from growing too large and causing numerical instability. In causal (left-to-right) models, such as predicting the next word in a sentence, or the next position in a trajectory, a mask is applied to ensure that tokens can only attend to the past, preventing the model from "cheating" by looking at future tokens during training [9].

Feedforward Neural Network. After the attention layer incorporates information across tokens, the Feedforward Network (FFN) processes each token position independently and in parallel [9]. It is usually a two-layer network with a non-linear activation such as ReLU:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 \tag{9}$$

This layer allows the model to perform more complex transformations of the contextualised information gathered by the attention heads [9].

Returning to the intuitive picture introduced earlier, the attention mechanism can be viewed as first updating each word representation using information from the other words in the sequence. The FFN then further refines each of these updated representations independently at each position, using transformations learned from many training sequences.

2.6.3 Prediction Head

After the data passes through the final transformer block, the final representations are passed to the next-token prediction head, which converts them into a probability dis-

tribution over possible next tokens in the sequence. This is done through the following steps.

Output Projection. Let $\mathbf{h}_t$ denote the final hidden representation produced after the model has processed the prefix up to position t. This d-dimensional vector summarises the information in the observed prefix and is then projected back into the vocabulary space, often by multiplying it by the transpose of the embedding matrix E^T [9].

Softmax. The resulting logits are passed through a softmax function to produce a probability distribution over the next token:

$$\mathbf{p}_{t+1} = \text{softmax}(\mathbf{h}_t E^T) \tag{10}$$

where $\mathbf{p}_{t+1}$ is the probability distribution over possible values of the next token z_{t+1} given the prefix $z_{\leq t}$ [9].

In general, this means that the model assigns a probability to each possible next token given the preceding trajectory prefix. In an NLP setting, where tokens correspond to words, this can be interpreted as the model estimating which word is most likely to come next in the sentence. The next token is then selected according to a decoding strategy such as greedy decoding, sampling, or beam search.

2.7 The Markov Property

In planning tasks, there is typically an environment state that evolves over time as actions are taken. The Markov property states that the probability distribution of the next state depends only on the current state and the action taken, and not on the history of states and actions that led to the current state [1]. In the previously discussed problem of finding a sequence of actions connecting a starting position to a goal position in a known maze, the Markov property holds if the current position, selected action, and maze layout are sufficient to determine the probability distribution over possible next positions.

If the maze layout is known and constant, and $(s_1, s_2, \ldots, s_t)$ are successive positions in the maze, then the state-transition process satisfies the Markov property if

$$p(s_t \mid s_1, a_1, s_2, a_2, \ldots, s_{t-1}, a_{t-1}) = p(s_t \mid s_{t-1}, a_{t-1}). \tag{11}$$

Transition dynamics describe how the state of an environment changes after an action is taken, or equivalently, the probability distribution of the next state given the current state and selected action. If the next position is uniquely determined by the current position and selected action, then there exists a transition function f such that

$$s_t = f(s_{t-1}, a_{t-1}). \tag{12}$$

The maze environment therefore has deterministic Markovian transition dynamics.

If the objective is to train a generative model that estimates a probability distribution over possible next actions for maze planning, it may be advantageous to condition the model on all previous states and actions. The model therefore predicts

$$p(a_t \mid m, s_{\text{start}}, s_{\text{goal}}, s_1, a_1, s_2, a_2, \ldots, a_{t-1}, s_t) \tag{13}$$

rather than

$$p(a_t \mid m, s_{\text{start}}, s_{\text{goal}}, s_t), \tag{14}$$

even if the state-transition process satisfies Equation (11).

The hypothesis is that previous states and actions can still provide useful information to the model when predicting the probability distribution over next actions. For example, if the model returns to a position that it has already visited, the trajectory prefix allows it to identify the earlier visit and may help it avoid repeating the same sequence of actions. The key distinction is that the Markov property describes the environment's transition dynamics, rather than the information on which a learned model may be conditioned. The fact that the next state is determined by the current state and selected action therefore does not prevent a learned sequence model from using previous states and actions when predicting the next-action distribution.

2.8 Fully- and Partially Observed States

The Markov property is a property of an environment or stochastic process. An agent navigating the environment, or a model predicting aspects of it, may assume that the Markov property holds. Under this assumption, the probability distribution of the next state depends only on the current state and selected action, rather than on the complete history of previous states and actions. If the complete current state is made available, the environment is then said to be fully observed from the perspective of the model. The complete states and the environment's transition dynamics being known, means the probability distribution of the next state after taking a particular action can be calculated. In practice, however, an agent or model may not have access to the complete state of the environment. Instead, only certain parts or aspects of that state may be observed. In this case, the environment is said to be partially observed from the perspective of the agent or model.

In the context of the maze planning problem, the maze layout being known implies that coordinates of the current position provide enough information to know the exact position within the maze, making the environment fully observed. If the model is however not provided with its exact coordinates in a maze, but rather with a local view of the maze layout, such as a 3x3 grid of cells surrounding the current position, then the model does not have enough information to know its exact position in the maze, as multiple positions in multiple maze layouts may correspond to the same local view. These local views are referred to as observations and are provided to the model instead of the exact states, making the environment partially observed. The environment can still be Markovian, but the current observation and action may not provide enough information to determine the next observation, so the observation process may not satisfy the Markov property. A model may need to rely on previous observations and actions to infer its current position in the maze.

3 Literature Review

3.1 Partial Observability and Conditioning on Observation History

Recall that planning is concerned with finding a sequence of actions that an agent living in an environment can take to reach a goal state from a starting state. A Deep Q-Network (DQN) uses a neural network to estimate the expected cumulative reward associated with each action from a given state or observation. Hausknecht and Stone [8] modify a DQN to include an LSTM that incorporates previously observed information, investigating whether this improves performance on Atari games.

As part of their study, Hausknecht and Stone investigate the performance of these modified networks on partially observed games. In such games, previous frames can provide information that is missing from the current observation. Their Deep Recurrent Q-Network outperformed a standard DQN on the partially observed Flickering Pong task, although they found no systematic advantage across the other games they examined [8].

Their findings suggest that past observations can be useful when the current observation does not provide enough information to identify the best action. Motivated by this, the partially observed experiment in the present study conditions a Transformer on a trajectory prefix containing both past observations and actions, allowing the model to use both when estimating probabilities over next moves.

3.2 Decoding Strategies for Sequence Generation

Sutskever et al. [4] provide an early example of beam search being used to generate sequences from a learned sequence model, aiming to maximise sequence likelihoods.

Sutskever et al. primarily investigate the use of long short-term memory (LSTM) networks, a type of recurrent neural network designed to capture long-range dependencies, for English-to-French translation while making minimal assumptions about the underlying sequence structure. Although they used LSTMs rather than Transformers and studied translation rather than planning, the process of using beam search to generate sequences remains similar in Sutskever et al.'s approach and the present study. They found that their model performed well even with a beam width of one, while a width of two provided most of the benefits of a wider beam [4].

Britz et al. [5] investigate how various parameters, such as embedding size, network depth, RNN cell type and residual connections, affect performance in machine translation tasks. In particular, one of the investigated parameters is the decoding heuristic.

Britz et al. fix the trained model and compare machine translation performance when beam search of various widths is used as the decoding heuristic, including a beam width of one, corresponding to greedy decoding in their experiments. They found that moderate beam widths improved translation performance relative to greedy decoding, but very wide beams reduced performance [5]. The present study utilises a similar experimental design to the decoding algorithm comparison by Britz et al., where a model and problem are

held fixed, and performance is measured across multiple decoding algorithms.

3.3 Planning as Sequence Modelling

Janner et al. [3] investigated how various planning problems can be solved by modelling states, actions and rewards related to each environment as a sequence of tokens and fitting a Transformer to autoregressively predict the next token in the sequence. This process is proposed as a method for offline reinforcement learning, challenging traditional methods such as conservative Q-learning (CQL).

Janner et al. utilised beam search as a planner, using it to generate trajectories from the learned model. They used likelihood-based beam search for imitation learning and goal-reaching, and reward-guided beam search for offline reinforcement learning [3].

Janner et al. found that the sequence-modelling view of reinforcement learning is effective. They also noted that prediction with their Transformer is slower and more resource-intensive than prediction with conventional models that estimate one environment transition at a time [3].

The present study is closely related to some of the work done by Janner et al., specifically in modelling states and actions or observations in a planning problem as a sequence, training a Transformer on many trajectory sequences, and generating final trajectories by decoding the trained Transformer. A fundamental difference is that Janner et al. aim to show whether the previously mentioned strategy is effective in solving reinforcement learning problems, without conducting an experiment investigating how decoding algorithms of varying breadths affect performance when sampling from a fixed model. In contrast, the present study investigates a planning problem in only one environment, and compares the performance of three decoding strategies.

The present study also does not include rewards in the training trajectories, and rewards are thus not learned and cannot be predicted by the trained model. The decoding algorithms therefore maximise the likelihoods predicted by the model, thus performing imitation learning. Janner et al. investigate both imitation learning and using the predicted rewards to determine decoding choices.

Furthermore, Janner et al. aim to solve a series of continuous planning problems by discretising the states, actions and rewards [3]. The problem investigated in the present study, however, is discrete by design.

3.4 Positioning of This Study

The previously mentioned papers show how planning problems can be approached through sequence modelling, demonstrate the use of beam search in sequence generation, investigate the effects of different beam widths, and provide evidence that observation histories can be beneficial in partially observed environments. The present study demonstrates an empirical procedure for comparing decoding strategies in a specific set of planning problems.

The partially and fully observed maze navigation problems explored in this research have many possible solutions and approaches. Even when attempting to solve the maze planning problems through autoregressive sequence modelling, there are many variables and design choices that affect the results, including but not limited to the data preprocessing, the choice of supervised learning model, the design of the model, the conditioning of the data provided to the model and how complete trajectories are generated using the trained model. Within each planning experiment, the decoding algorithms are compared using the same trained model and evaluation queries.

The present study therefore does not aim to find a state-of-the-art solution to the particular maze planning problems, nor does it attempt to contribute new theory on decoding or sequence modelling. The study aims to develop an understanding of how a planning problem can be solved through sequence modelling and how the choice of decoding strategy affects goal-reaching success, steps until termination, and computational cost.

4 Research Questions

The objective of this work is to compare different decoding strategies when modelling maze planning as autoregressive inference in a learned conditional trajectory model trained through next-token prediction. The performance of greedy decoding and beam search with different beam widths are evaluated in terms of trajectory validity, goal-reaching success, steps until termination, and computational cost.

Four experiments were conducted to compare the decoding strategies. The first experiment is conducted in a maze environment where the maze indentity as well as the exact coordinates of the goal position, starting position and subsequent states are provided. Decoding algorithms are thus evaluated in a fully observable setting, where the model has access to all relevant information about the environment.

The second experiment utilises the same maze layouts and training trajectories as the first, but the model is not provided with the maze layout or the exact coordinates of the goal position. The model is instead provided with the relative direction of the goal position, and a layout of the 3x3 grid surrounding the current position istead of the complete states. This formulation is therefore a partially observable setting, and the model will need to rely on previously observed trajectory information to infer unobserved information about the maze layout.

Both of the preceding experiments compare decoding strategies using models trained on datasets of optimal trajectories. A third experiment uses the same partially observed setting as the second experiment, but trains the model on a larger dataset containing a mixture of optimal, suboptimal, and unsuccessful trajectories. This experiment investigates how decoding performance changes when the training data contains multiple trajectories of varying quality.

A final experiment modifies the decoding algorithms used with the model trained on the mixed-quality dataset in the partially observed setting. Rather than ranking candidate trajectories solely by their model-predicted likelihoods, the algorithms also apply an inference-time bonus to actions favoured by a predefined local rule. This introduces an

inductive bias: an assumption about which moves are likely to be productive, without retraining the model. This experiment tests whether changing only the decoding score improves the quality of the generated trajectories. Janner et al. [3] use a different form of reward-guided decoding: their model is trained to predict rewards and reward-to-go, and beam search uses model likelihoods to generate candidate transitions before ranking trajectories by cumulative predicted reward and a reward-to-go estimate. The mixed-quality dataset used here contains no reward or return targets. Following their approach would therefore require defining a reward signal, augmenting the training data, and training another model. The final experiment instead calculates a local bonus during decoding. However, a poorly chosen rule or bonus strength could reduce decoding performance.

5 Fully Observed Planning

5.1 Problem Formulation

The experiments to follow evaluate different decoding strategies by training a conditional trajectory model through next-token prediction in a fully observed maze environment. Given a maze layout identifier, a start state, a goal state, and a trajectory prefix consisting of previous actions and explicit coordinates of previous states in the maze, the model predicts the next action in the trajectory sequence. At inference time, decoding algorithms are then used to construct complete trajectories by selecting continuations with high probability under the learned model. Since decoding is based on model probability rather than a reward-maximising objective, this amounts to using imitation learning to solve the planning problem.

A trajectory $\boldsymbol{\tau} = (z_1, z_2, \ldots, z_{2T})$ consists of alternating state and action tokens, $(s_1, a_1, s_2, a_2, \ldots, s_T, a_T)$, where the states represent coordinates in the maze and the actions correspond to the moves (up, down, left, or right) between consecutive states. Thus, $z_{2t-1} = s_t$ and $z_{2t} = a_t$ for $t = 1, \ldots, T$: odd-indexed tokens are states and even-indexed tokens are actions. In this environment, the state immediately following an action is fully determined by that action and the preceding state. Supervision is therefore applied only at action-token positions, so the learning problem is more precisely to predict the next action conditioned on the maze layout, the start state, the goal state, and the previously observed trajectory prefix.

Let m denote a maze layout identifier, let s_{start} and s_{goal} denote the start and goal states. The model therefore learns conditional predictions of the form

$$p_{\hat{\boldsymbol{\theta}}}(a_t \mid m, s_{\text{start}}, s_{\text{goal}}, s_1, a_1, \ldots, s_t). \tag{15}$$

These conditional predictions define an autoregressive factorisation of the conditional distribution over action sequences:

$$p_{\hat{\boldsymbol{\theta}}}(a_1, a_2, \ldots, a_T \mid m, s_{\text{start}}, s_{\text{goal}}) = \prod_{t=1}^{T} p_{\hat{\boldsymbol{\theta}}}(a_t \mid m, s_{\text{start}}, s_{\text{goal}}, s_1, a_1, \ldots, s_t). \tag{16}$$

Ideally, generating a complete plan would require sampling from the full joint conditional

distribution over actions and observations above; in practice, the decoding algorithms instead operate on its autoregressive conditional factors.

A Transformer was selected as the generative model because of its demonstrated performance on sequence-modelling tasks [13], the wide range of Python libraries available for working with it, and its computational efficiency when the architecture is kept small. The research question, however, is not whether a Transformer can represent such a distribution in principle, but how different decoding strategies affect the quality of the generated plans once the model has been trained. In particular, the methodology compares the performance of greedy decoding and beam search in a fully observed maze environment. The performance is measured in terms of trajectory validity, goal-reaching success, steps until termination, and computational cost.

5.2 Methodology

5.2.1 Dataset Construction

A trajectory token sequence provides multiple supervised training examples, since for each token position k in a given trajectory, the prefix

$$(z_1, z_2, \ldots, z_{k-1}) \tag{17}$$

can be used to predict the next token z_k. In addition to the trajectory tokens themselves, the model input must encode which maze the trajectory belongs to, as well as the associated start and goal positions.

This work considers eight maze layouts from the D4RL PointMaze/Maze2D environment. The exact layout definitions are provided in the D4RL repository's `maze_model.py` file [12]. Figure 2 shows the eight layouts, which vary in size and structural complexity. This variation allows the decoding strategies to be evaluated across a range of planning problems.

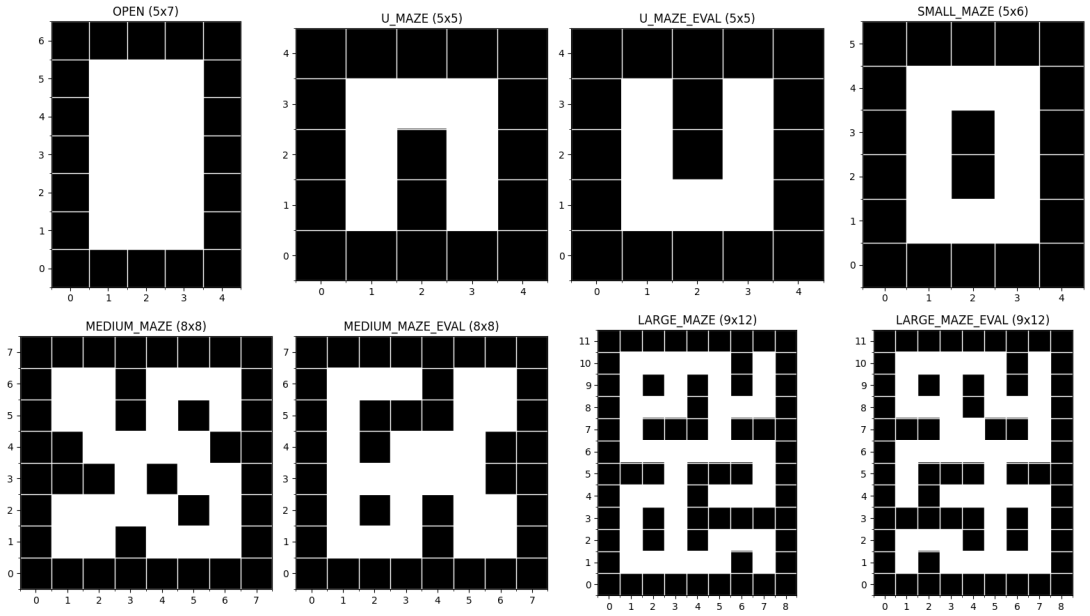


Figure 2: Maze layouts used from the D4RL PointMaze / Maze2D environment.

Each maze is defined as a matrix, and the row and column indices of this matrix determine the corresponding grid cells in the discrete maze representations used throughout this work. To represent the states, actions, and maze layouts as Transformer inputs, each is encoded as a discrete integer token. Distinct token ranges are reserved for different token classes by defining token offsets, so that maze tokens, state tokens, and action tokens occupy non-overlapping regions of the overall vocabulary.

States are encoded by assigning each grid location a unique integer index. This is done using the mapping

$$\text{state_index} = \text{row} \times (\text{max columns}) + \text{column}, \tag{18}$$

where *row* and *column* are the integer indices of the grid location, and *max columns* is the maximum number of columns across all maze layouts. This produces a unique integer identifier for each location in the global grid. A state token is then obtained by adding a fixed state-token offset to this index. Although this indexing scheme assigns identifiers to all grid locations in the global grid, including wall locations, only valid free-cell positions appear in the generated trajectory sequences.

Actions are also encoded as integer tokens. Since the trajectories are restricted to four-connected grid movements, the action vocabulary consists of the four moves up, down, left, or right, with each assigned its own unique integer token identifier.

In the training data, the detailed wall structure of a maze is not included explicitly in the token sequence. Instead, each trajectory is associated with a single integer maze token identifying which of the eight layouts it belongs to. The model must infer the relevant layout constraints from this maze identifier together with the observed trajectory tokens. This design choice gives a simple and efficient encoding scheme, but it also means that the model is tied to the set of maze layouts seen during training and cannot directly generalise to previously unseen layouts without additional representation of maze geometry. Although the maze geometry is not included explicitly in the token sequence, the

maze identifier uniquely specifies one of the eight fixed layouts used in the experiment. Together with the current and goal coordinates, it therefore uniquely identifies the complete planning situation within this restricted set of mazes. The problem is consequently treated as fully observed: any uncertainty caused by the model learning the association between a maze identifier and its layout imperfectly is a limitation of the learned model rather than a consequence of partial observability.

The tokenised input sequences are stored in an input matrix X of dimensions $N \times L$, where N is the number of training trajectories and L is the maximum token-sequence length observed in the training dataset. The final form of each row $\boldsymbol{x}$ of the input matrix X is

$$\boldsymbol{x} = (m,\ s_{\text{start}},\ s_{\text{goal}},\ s_1,\ a_1,\ s_2,\ a_2,\ s_3,\ \ldots,\ a_T,\ s_{T+1},\ 0,\ \ldots,\ 0). \tag{19}$$

where s_{start} and s_{goal} are represented using the same coordinate encoding as the states appearing in the trajectory. The sequence $(s_1, a_1, s_2, a_2, \ldots, s_T, a_T)$ represents the alternating state and action tokens that make up the trajectory. Any unused entries at the end of the row are filled with padding tokens (0) so that all rows of X have the same length.

For each maze layout, every valid ordered pair of distinct, non-wall start and goal positions was considered. Breadth-first search (BFS) was used to find a shortest path connecting each pair. The positions along each resulting path, together with the actions corresponding to the movements between consecutive positions, were stored as complete state-action trajectories for training the Transformer.

To compare decoding strategies on unseen queries, the transformer-ready dataset is partitioned using 5-fold cross-validation. The dataset contains exactly one trajectory for each ordered start-goal pair that is connected by a valid shortest path in a given maze. Within each maze, trajectories were ordered lexicographically by path length, start row, start column, goal row, and goal column. They were then assigned to the five folds in round-robin order: the first trajectory was assigned to fold 1, the second to fold 2, and so on, returning to fold 1 after every fifth trajectory. This deterministic procedure was chosen to make the cross-validation split reproducible. Because each start-goal pair appears only once in the dataset, no exact query appears in both the training and evaluation portions of a given fold. In the decoding benchmark, evaluation is reported on the eight maze families as seen in Figure 2. The aim of the experiment is therefore to compare how different decoding rules behave on unseen start-goal planning queries under a fixed trained model class.

5.2.2 Transformer Training

In addition to the input matrix X, the token-type information is stored in a token-type matrix T, the target outputs are stored in a label matrix Y, and the padding information is stored in a mask matrix M. T, Y and M are all of the same shape as X. Each entry in the token-type matrix indicates whether the corresponding token is a maze token, start token, goal token, state token, action token, or padding token. In the label matrix, only action-token positions contain active targets, while all other positions are ignored during loss computation. The mask matrix indicates which positions in the input sequences are padding tokens.

A small causal Transformer was trained on the tokenised trajectory datasets. After the model receives the training data, token embeddings and token-type embeddings are summed, positional encodings are added, and the resulting sequence is processed by a stack of causal self-attention layers so that each token can depend only on the current and preceding tokens in the sequence.

The model was kept small for faster training. The architecture used a hidden dimension of 64, 4 attention heads, 2 Transformer layers, a feedforward dimension of 128, and dropout of 0.1. Training was performed using the AdamW optimiser with learning rate 3×10^{-4} and weight decay 10^{-4}. For each training fold, the model was trained for 20 epochs on the training partition of the fold, with a batch size of 128. After every epoch, the model was evaluated on both the training and validation partitions of the fold. The checkpoint with the best validation loss was saved and used for decoding evaluation.

The training objective was cross-entropy loss applied to next-token prediction, with supervision active only at action-token positions. This corresponds to modelling each conditional action distribution as categorical over a finite vocabulary, for which cross-entropy is the negative log-likelihood objective. Non-action positions in the label matrix Y were ignored during loss computation.

5.2.3 Decoding

The decoding strategies compared in this work are greedy decoding and beam search with beam widths 2 and 3. The trained Transformer predicts a distribution over the next action token conditioned on the maze token, start token, goal token, and the trajectory prefix generated so far. Decoding then determines how these conditional predictions are converted into a complete trajectory.

For a given evaluation query, decoding begins from the token prefix consisting of the maze identifier, the start token, the goal token, and the initial state token. The model is then queried autoregressively for the next action token. Candidated actions are restricted to be valid in the given maze environment. Once an action has been selected, the corresponding next state is deterministically computed from the current state and appended to the trajectory prefix, and the procedure repeats until the goal is reached or a step limit is exceeded. This procedure assumes that the environment's transition dynamics are known to the planner. Since these dynamics are deterministic, the planner can calculate the next position exactly from the maze layout, current position, and selected action, rather than requiring the model to infer the transition dynamics or predict a probability distribution over possible next positions from the trajectory prefix.

The step limit refers to the maximum number of decoding steps, equivalently the maximum number of action predictions, that a decoding algorithm is allowed to generate for a query before termination is forced if the goal has not yet been reached. In the implementation used here, this limit is fixed at 19 steps for all queries. This value was chosen as it is the maximum number of actions to reach the goal found in the training dataset across all maze layouts.

5.2.4 Evaluation Metrics

Since the Transformer only provides an estimated conditional distribution over next actions, a decoding strategy may fail to reach the goal within the allowed horizon even when a valid path exists. For this reason, the first evaluation metric is completion rate, defined as the proportion of evaluation queries for which the decoded trajectory reaches the goal within the allowed number of steps.

The second metric is average steps until termination. This is computed as the mean number of decoding steps over all evaluation queries. A successful query terminates when the goal is reached, whereas an unsuccessful query terminates at the 19-step limit. This metric therefore reflects both goal-reaching success and the number of steps taken; a lower value does not by itself establish that successful paths are shorter.

The remaining two metrics quantify computational cost. For each decoded trajectory, the total wall-clock time required to perform decoding is recorded. This yields average Compute Seconds, defined as the mean total decoding time per evaluation query. In addition, the total decoding time for a query is divided by the number of decoded actions taken for that query, yielding a per-query seconds per step quantity. Averaging this value over evaluation queries gives average seconds per step. Together, these four metrics allow the comparison to capture both planning quality and computational efficiency.

5.3 Results and Discussion

Table 1 and Table 2 report decoding performance across the eight fully observed maze layouts, aggregated over the full cross-validation evaluation. The main text focuses on completion rate and average steps until termination. The full set of results, including completion rate, average steps until termination, average compute time, and average time per step, is reported in Appendix B.

Table 1: Completion rate.

Maze layout	Greedy	Beam 2	Beam 3
`open`	0.8292	0.9708	1.0000
`u_maze`	0.7333	0.9778	1.0000
`u_maze_eval`	0.8667	1.0000	1.0000
`small_maze`	1.0000	1.0000	1.0000
`medium_maze`	0.7492	0.9446	0.9877
`medium_maze_eval`	0.6292	0.9077	0.9723
`large_maze`	0.6657	0.9213	0.9845
`large_maze_eval`	0.6901	0.8992	0.9699

Table 2: Average steps until termination.

Maze layout	Greedy	Beam 2	Beam 3
`open`	8.0417	4.4292	3.3750
`u_maze`	8.0111	4.0556	3.6889
`u_maze_eval`	6.3222	3.6667	3.6667
`small_maze`	2.2500	1.6667	1.6667
`medium_maze`	9.6185	6.1631	5.3246
`medium_maze_eval`	10.8523	6.5815	5.6123
`large_maze`	12.0778	9.0691	8.1986
`large_maze_eval`	11.7229	9.0037	8.0014

It is observed in Tables 1 and 2 that beam search with width $B = 2$ and $B = 3$ outperforms greedy decoding in terms of both completion rate and average steps until termination, with the larger beam width yielding the best overall performance. This is consistent with the expectation that a broader search over candidate trajectories yields trajectories with higher probability under the learned model, and since the model was trained on optimal trajectories, these higher-probability trajectories are more likely to correspond to valid and efficient paths to the goal.

The trade-off typically associated with beam search, relative to greedy decoding, and with increasing beam width, is that the broader search requires more computation. In the aggregate, the average time per step is indeed higher for beam search than for greedy decoding, and it also increases with beam width. However, in this implementation the average total compute time per query is lower for beam search than for greedy decoding, and is also lower for $B = 3$ than for $B = 2$. This can be explained by the fact that broader searches more often reach the goal before the step limit, so the reduction in the number of decoded steps outweighs the increase in time per step, yielding a lower total compute time.

The performance of the decoding strategies did vary across maze layouts. However, the relative ordering in completion rate and average steps until termination was consistent across all maze layouts: beam search performed at least as well as greedy decoding, and $B = 3$ performed at least as well as $B = 2$. For average time per step, the same overall pattern also held in almost all cases. The only exception was the U_MAZE layout, where beam search with width $B = 3$ had a slightly lower average time per step than beam search with width $B = 2$.

In some mazes, the average total compute time did not decrease monotonically with beam width. This occurred in layouts where there were typically fewer valid path options from a given state, so the average steps until termination of the broader searches were close to, or equal to, those of the narrower searches. In those cases, the additional per-step cost of the broader search was not always offset by a shorter decoded trajectory.

The results therefore demonstrate that beam search is a more effective decoding strategy than greedy decoding for the maze planning problem under the learned trajectory model, both in terms of goal-reaching success and average steps until termination. The strongest results were obtained with beam width $B = 3$, which achieved the highest completion rate and the lowest average steps until termination across the full evaluation.

6 Partially Observed Planning

6.1 Problem Formulation

The objective of this experiment is to evaluate different decoding strategies for a conditional trajectory model trained through next-token prediction under a partially observed setting. At each time step, the model is provided with neither the maze layout nor the absolute coordinates of the agent or goal. Instead, it receives a 3×3 grid of cells centred on its current position and the displacement of the goal relative to that position. The relative displacement provides the row and column offsets of the goal, including their directions, without revealing the agent's absolute position or the unobserved maze layout. It does not provide the shortest-path distance through the maze.

Unlike in the fully observed experiment, where the complete maze layout is available, the preceding trajectory tokens now contain information that may not be present in the current local observation. Figure 3 contrasts the complete maze layout and the agent's position with the local observation at a given time step. The right panel represents all the

information supplied to the model at that time step. In this partially observed setting, a planning algorithm does not have access to the true observations that would follow a sequence of actions without executing those actions in the environment. The planner can still compare possible future trajectories, but it must construct them using the trajectory information available so far and the model's predicted observations rather than the true future observations. This restriction did not arise in the fully observed experiment because the complete maze layout and deterministic movements made it possible to calculate the exact sequence of states resulting from any candidate action sequence without executing it in the environment.

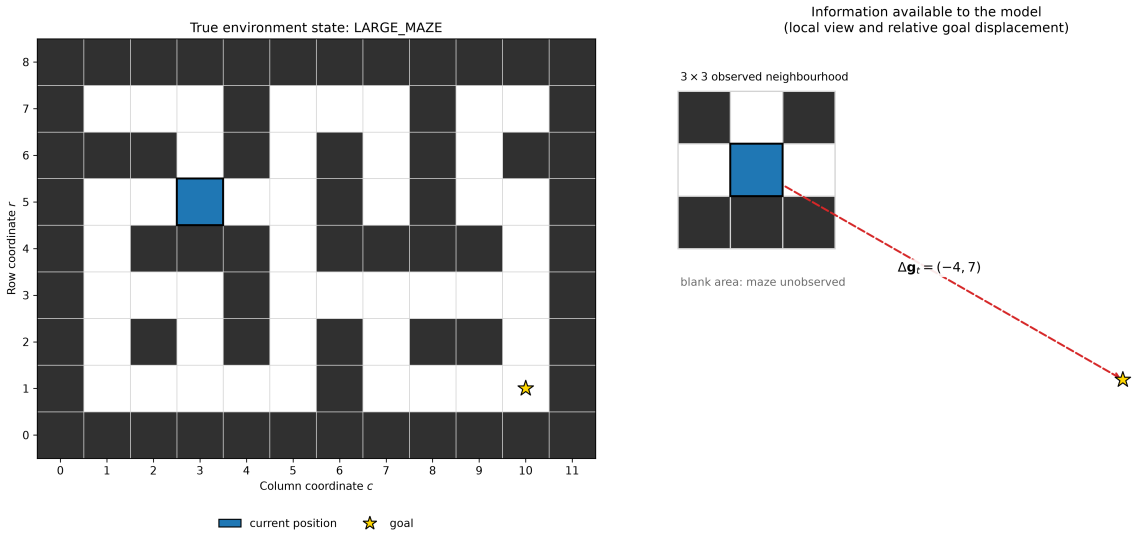


Figure 3: Comparison of the true maze state and the corresponding partial observation at the same time step. The left panel shows the full `LARGE_MAZE` layout, the agent's current position, and the goal using absolute grid coordinates. The right panel shows only the local 3×3 neighbourhood and the goal's relative displacement.

Conditioning on the complete trajectory prefix allows the Transformer to use previous observations and actions to infer unobserved aspects of the maze layout. The partially observed setting therefore allows the decoding strategies to be compared in a problem where an advantage of the Transformer-based sequence-modelling framework is particularly relevant: its ability to use trajectory context when the current observation alone is insufficient for selecting the next action.

At inference time, the same decoding algorithms as in the previous experiment are used to construct complete trajectories by selecting continuations with high probability under the learned model. As in the previous experiment, the model is trained by imitating demonstrated trajectories, and decoding is based on model probability rather than reward maximisation. The action-selection component initially remains based on imitation learning, and observation prediction allows beam search to evaluate imagined continuations, which refer to predicted values of unknown future observations that would follow candidate action sequences.

A modified version of the decoding algorthim where a predefined rule influence action choices is also investigated. This rule alongside model probability determine what candi-

date trajectories is preferred by the decoders.

During decoding, beam search needs to form a set of candidate trajectories. In the previous experiment, beam search could calculate the exact trajectories corresponding to each candidate action sequence, since the exact position in the maze after every action is known. In the partially observed setting, however, the observations that would follow hypothetical actions are not known, so they must be predicted. Supervision during training is thus applied to the observations as well as the actions in the sequences.

Let o_t denote the local observation at time step t, consisting of the 3×3 grid of cells surrounding the agent's current position. The position of the goal is provided separately as the relative displacement

$$\Delta \mathbf{g}_t = \begin{pmatrix} r_{\text{goal}} - r_t \\ c_{\text{goal}} - c_t \end{pmatrix}, \tag{20}$$

where (r_t, c_t) denotes the agent's current position and $(r_{\text{goal}}, c_{\text{goal}})$ denotes the goal position. For notational convenience, the complete observation supplied at time step t is denoted by

$$\mathbf{y}_t = (o_t, \Delta \mathbf{g}_t). \tag{21}$$

A trajectory containing T observations therefore has the form

$$\boldsymbol{\tau} = (\mathbf{y}_1, a_1, \mathbf{y}_2, a_2, \ldots, \mathbf{y}_{T-1}, a_{T-1}, \mathbf{y}_T), \tag{22}$$

where each action a_t is one of the four movements (up, down, left, or right) performed between observations $\mathbf{y}_t$ and $\mathbf{y}_{t+1}$. The model is not provided with a maze identifier, the complete maze layout, or the absolute coordinates of the agent or goal.

The first observation $\mathbf{y}_1$ is supplied to the model. Supervision is then applied to each action and each post-action observation. The model consequently learns conditional predictions of the forms

$$p_{\hat{\boldsymbol{\theta}}}(a_t \mid \mathbf{y}_1, a_1, \mathbf{y}_2, a_2, \ldots, a_{t-1}, \mathbf{y}_t), \tag{23}$$

and

$$p_{\hat{\boldsymbol{\theta}}}(\mathbf{y}_{t+1} \mid \mathbf{y}_1, a_1, \mathbf{y}_2, a_2, \ldots, \mathbf{y}_t, a_t). \tag{24}$$

Since $\mathbf{y}_{t+1}$ is represented by multiple tokens, its components are themselves predicted autoregressively in their token order.

These conditional predictions define an autoregressive factorisation of the joint conditional distribution over future actions and observations:

$$\begin{aligned} &p_{\hat{\boldsymbol{\theta}}}(a_1, \mathbf{y}_2, a_2, \ldots, a_{T-1}, \mathbf{y}_T \mid \mathbf{y}_1) \\ &\quad = \prod_{t=1}^{T-1} p_{\hat{\boldsymbol{\theta}}}(a_t \mid \mathbf{y}_1, a_1, \ldots, a_{t-1}, \mathbf{y}_t) p_{\hat{\boldsymbol{\theta}}}(\mathbf{y}_{t+1} \mid \mathbf{y}_1, a_1, \ldots, \mathbf{y}_t, a_t). \end{aligned} \tag{25}$$

6.2 Methodology

6.2.1 Dataset Construction

The original partially observed dataset is constructed from the same eight mazes and the same optimal BFS trajectories as the fully observed dataset. The ordered start-goal queries, action sequences, path lengths, episode identifiers, maze assignments, and

five-fold split assignments are retained unchanged. The trajectories are therefore not generated again; instead, the fully observed state representation is replaced by the partial observation representation defined above. This preserves a controlled comparison between the experiments while avoiding repetition of the trajectory-generation and partitioning procedures.

For the additional mixed-quality experiment, a second dataset is constructed using the same start-goal queries and fold assignments. Three trajectories are generated independently for each query. At every step, the trajectory-generation algorithm selects a valid action that reduces the shortest-path distance to the goal with probability 0.8, or selects a random valid action with probability 0.2. A trajectory ends when it reaches the goal or after 19 actions. The random choices allow the resulting trajectories to be optimal, suboptimal, or unsuccessful, and all three types are retained. This produces 17,928 trajectories in total. The shortest-path distances are used only to generate the training trajectories and are not provided to the Transformer.

At each position in both datasets, the local observation o_t is encoded using two discrete tokens. The first is a nine-bit wall mask w_t, with one bit for each cell in the 3×3 neighbourhood; cells containing walls or lying outside the maze boundary are marked as blocked. The neighbourhood retains the fixed orientation of the maze, and the bits are ordered from left to right across the top row, followed by the middle row and then the bottom row. The second token v_t indicates the local position of the goal when it is visible within this neighbourhood, or indicates that the goal is not locally visible. The two components of the relative goal displacement, Δr_t and Δc_t, are each represented by a separate discrete token. Actions use the same four-action vocabulary and integer-token encoding as in the fully observed experiment, and distinct token ranges are again reserved for the different token classes.

In terms of these tokens, the local observation and complete model observation are therefore

$$o_t = (w_t, v_t), \qquad \mathbf{y}_t = (w_t, v_t, \Delta r_t, \Delta c_t). \tag{26}$$

For clarity, the following representation shows only the trajectory tokens and padding. Each row $\boldsymbol{x}$ of the input matrix has the form

$$\boldsymbol{x} = (w_1, v_1, \Delta r_1, \Delta c_1, a_1, w_2, v_2, \Delta r_2, \Delta c_2, a_2, \ldots, a_{T-1}, w_T, v_T, \Delta r_T, \Delta c_T, 0, \ldots, 0). \tag{27}$$

As before, the tokenised sequences are stored in an $N \times L$ input matrix and unused positions are filled with padding tokens. Unlike the fully observed representation, these rows contain no maze-identifier token, absolute start or goal coordinates, or absolute state coordinates.

The initial observation is supplied as context. Supervision is applied at every action position and to every component of each post-action observation. Thus, for each transition, the supervised targets are the action a_t and the tokens $(w_{t+1}, v_{t+1}, \Delta r_{t+1}, \Delta c_{t+1})$. The initial observation and padding positions are excluded from the training loss.

6.2.2 Transformer Training

The same causal Transformer design and optimisation procedure as in the fully observed experiment are used, although the vocabulary size and maximum sequence length are ad-

justed for the new token representation. The hidden dimension, attention heads, Transformer layers, feedforward dimension, dropout, optimiser, learning rate, weight decay, batch size, and 20-epoch training limit are unchanged. Supervision is also applied to both actions and post-action observations. Consequently, rather than repeating training for the complete five-fold cross-validation procedure, folds 1–4 are used for training, while fold 0 is used both for validation during checkpoint selection and for the final decoding evaluation. As before, the checkpoint with the lowest validation loss is retained for decoding evaluation.

The input, token-type, label, and padding-mask matrices are constructed in the same manner as before, but the trajectory-token types now identify wall-mask, visible-goal, goal-displacement, and action tokens. The loss remains categorical cross-entropy for causal next-token prediction, but its supervision mask is expanded. Active targets are placed at every action-token position and at each of the four tokens representing the subsequent observation. The same Transformer is therefore trained jointly to predict a_t and $(w_{t+1}, v_{t+1}, \Delta r_{t+1}, \Delta c_{t+1})$ from the preceding trajectory prefix. The initial observation and padding positions remain context only and do not contribute to the loss.

6.2.3 Decoding

The same greedy and beam-search strategies, beam widths of 2 and 3, and 19-action limit as in the fully observed experiment are used. Unlike in the previous experiment, however, the decoding algorithms cannot, in general, determine the exact observations that would result from actions taken multiple steps into the future without first taking those actions in the environment. Beam search therefore cannot construct and compare complete candidate trajectories using exact resulting observations, as it could in the previous experiment. Beam search was therefore modified to use the fitted model to predict both actions and the observations resulting from those actions, allowing it to construct candidate continuations containing up to five future actions and predicted observations. After each token is generated, all one-token extensions of the current candidates are ranked using their accumulated log-probabilities, and only the 2 or 3 highest-scoring prefixes are retained. Beam search therefore applies the same token-level pruning procedure throughout the generated action and observation sequence.

When a candidate trajectory is extended, possible actions are restricted using the wall mask associated with the final observation in that candidate trajectory. For the first action, this wall mask comes from the observation received from the environment. For later imagined actions, it comes from an observation predicted by the model. Each retained action prefix is then extended with predicted wall-mask and visible-goal tokens, with the beam pruned after each token is added. The relative goal displacement is updated directly from the previous displacement and the candidate action, while the log-probabilities assigned by the model to the corresponding row- and column-displacement tokens contribute to the trajectory score and beam ranking. Later actions in a candidate trajectory are therefore selected using predicted local observations, without consulting the complete maze layout.

After a beam-search continuation has been selected, only its first action is taken in the maze. The remainder of the predicted continuation is discarded, the resulting local obser-

vation is added to the trajectory prefix, and a new five-step continuation is constructed. This process of repeatedly planning several steps ahead but taking only the first action is referred to as receding-horizon planning. The predicted observations are therefore not added to the trajectory prefix used at later positions. For both greedy decoding and beam search, decoding terminates when the goal is reached or when the 19-action limit is reached.

Greedy decoding does not need to compare the outcomes of multiple action sequences, so it only needs to predict the next action to take at the current position. It selects the highest-probability valid action, takes that action in the maze, receives the resulting observation, and repeats. Predicting a longer continuation is unnecessary because greedy decoding does not retain alternative first actions, so later predictions cannot influence the selected action.

An additional experiment uses the model trained on the mixed-quality dataset without retraining it, but adds a local bonus to the decoding scores. Using the available 3×3 wall mask and goal displacement, the rule favours valid actions that minimise the Manhattan distance from the next square to the goal. Ties are resolved by considering the closest free square reachable from each candidate within the observed neighbourhood. Actions favoured by this rule receive a bonus of λ in addition to their model-predicted log-probability. Greedy decoding uses this combined score to select its next action, while beam search adds the bonus whenever a candidate trajectory is extended by a favoured action. For imagined future steps, the rule uses the model's predicted wall masks. The bonus is calculated during decoding rather than predicted by the model, and $\lambda \in \{0, 0.25, 0.5, 1, 2\}$ is evaluated, with $\lambda = 0$ giving the original decoding rule.

The rule is based on the intuition that, without prior information about the maze layouts or training trajectories, initially choosing valid moves that minimise the Manhattan distance to the goal is a reasonable strategy until more of the maze is observed. In this work, however, the model has learned information about the training maze layouts and action patterns from the training data. The bonus is intended to favour locally promising moves when candidate actions or trajectories have similar model scores, although a larger bonus can also override more substantial score differences. It is hypothesised to be especially useful for the first few actions, before the observed trajectory has revealed much of the current maze layout.

6.2.4 Grid-Bug2 Baseline

Grid-Bug2 is included as a deterministic, non-learning baseline. It receives the same current 3×3 observation and relative goal displacement as the model. It attempts to move along a direct grid line connecting the initial position to the goal, when blocked, follows the obstacle boundary until it can return to that line closer to the goal. The algorithm selects and executes one action at a time and is evaluated on the same fold-0 queries with the same 19-action limit as the decoding strategies.

6.2.5 Evaluation Metrics

The decoding strategies are evaluated using the same metrics as in the fully observed experiment: completion rate, average steps until termination over all queries, average compute time per query, and average compute time per executed step. These metrics respectively measure how often the goal is reached within the 19-action limit, how many steps are taken before termination, and the computational cost of each decoding strategy.

6.3 Results and Discussion

6.3.1 Optimal-Trajectory Training

Tables 3 and 4 report performance on the fold-0 evaluation queries across the eight maze layouts. The model was trained on folds 1–4, as described above, using only optimal trajectories. The comparison focuses on completion rate and average steps until termination. The complete results, including the computational-cost metrics, are reported in Appendix C. Grid-Bug2, a deterministic algorithm for navigating towards a known goal using local observations, is included as a non-learning baseline.

Table 3: Completion rate on the partially observed fold-0 evaluation queries.

Maze Layout	Grid-Bug2	Greedy	Beam 2	Beam 3
OPEN	1.0000	1.0000	1.0000	1.0000
U_MAZE	1.0000	0.9444	0.9444	0.9444
U_MAZE_EVAL	1.0000	1.0000	0.8889	0.8889
SMALL_MAZE	1.0000	1.0000	1.0000	1.0000
MEDIUM_MAZE	0.6308	0.9308	0.9462	0.9385
MEDIUM_MAZE_EVAL	0.6538	0.8538	0.8538	0.8462
LARGE_MAZE	0.3454	0.8671	0.8333	0.8043
LARGE_MAZE_EVAL	0.3395	0.8453	0.8568	0.8406

Table 4: Average steps until termination over all partially observed fold-0 queries.

Maze Layout	Grid-Bug2	Greedy	Beam 2	Beam 3
OPEN	2.6250	2.6667	2.6250	2.6667
U_MAZE	4.6667	4.8333	4.9444	4.9444
U_MAZE_EVAL	5.5556	4.3333	4.8333	4.8333
SMALL_MAZE	1.6000	1.6000	1.6000	1.6000
MEDIUM_MAZE	11.1538	6.6462	6.1923	6.2385
MEDIUM_MAZE_EVAL	10.7231	6.9692	6.9923	7.0000
LARGE_MAZE	15.3406	9.3889	9.6353	9.8961
LARGE_MAZE_EVAL	14.7529	9.5081	9.2702	9.3857

The three decoding strategies yielded similar completion rates across the eight maze layouts, and beam search with width $B = 3$ performed slightly worse than the other two strategies in most layouts. The three decoding strategies also yielded similar average

steps until termination in the smaller maze layouts, but differed more in the larger layouts. Wider searches had longer average compute times and longer average time per step across all maze layouts. The performance of Grid-Bug2 in terms of completion rate and average steps until termination was competitive with the decoding strategies in the OPEN, U_MAZE, U_MAZE_EVAL, and SMALL_MAZE layouts, but its performance degraded more sharply than that of the decoding strategies as the maze layouts became more complex.

The results indicate that retaining and comparing more candidate trajectory prefixes did not provide a significant advantage over greedy decoding. This result in isolation could indicate that the current framework of training a Transformer on optimal trajectories and decoding based on model probability is not effective in the partially observed setting, and the results represent blindly selecting valid actions in the maze without any meaningful guidance from the model. However, the fact that the decoding strategies outperformed Grid-Bug2 in the more complex maze layouts is consistent with the model providing some useful guidance for action selection. This result does not prove that the learned guidance caused the improvement, because Grid-Bug2 may simply be less well suited to the more complex layouts.

Another hypothesis for the lack of performance improvement with beam search is that the model is not able to accurately predict the observation tokens associated with imagined future actions, so a candidate trajectory contains predicted observations that are not consistent with the true maze layout. Considering more candidate trajectories therefore does not provide useful information for selecting the next action over simply selecting the highest-probability valid action at the current position.

In the current experiment, the model performed poorly when predicting the first wall mask associated with the selected candidate trajectory at each real decision step. The wall-mask accuracy was 0.3347 for beam search with width $B = 2$ and 0.3202 for beam search with width $B = 3$. These values were obtained from the token-by-token beam-search evaluation on the fold-0 queries. The metric is defined as

$$\text{WallMaskAccuracy} = \frac{1}{N_{\text{steps}}} \sum_{t=1}^{N_{\text{steps}}} \mathbf{1}\left\{\hat{w}_{t+1} = w_{t+1}\right\}, \tag{28}$$

where N_{steps} is the total number of decoded steps across all evaluation queries, $\hat{w}_{t+1}$ is the first predicted wall mask in the candidate trajectory selected by beam search at step t, and w_{t+1} is the true wall mask at the resulting position at time $t+1$, revealed after executing the selected action in the environment. This low accuracy suggests that the model often fails to predict the local wall configuration that results from the executed action. The metric only evaluates the first predicted wall mask in the selected candidate trajectory; it does not measure the accuracy of every observation in every imagined trajectory. It is therefore reasonable to hypothesise that errors may accumulate over multiple imagined steps, but the wall-mask accuracy alone does not establish that multi-step predictions are less accurate.

6.3.2 Observation-Prediction Diagnostic

To investigate whether inaccurate observation prediction contributes to the lack of performance improvement with beam search, an additional experiment compares beam search with a modified version called *Oracle Beam*. After each imagined candidate action, Oracle Beam uses the hidden maze simulator to obtain the complete true next observation instead of using an observation predicted by the model. Future actions in each candidate trajectory are therefore selected using the true simulated observations. Oracle Beam has access to information that is unavailable under partial observability, so it violates the constraints of the current planning problem and should be treated only as a diagnostic comparison, rather than as a valid solution to the planning problem.

Because the observation tokens used by Oracle Beam are supplied by the simulator rather than selected from the model's predictions, their probabilities are not included in the accumulated log-probability of a candidate trajectory. Candidate trajectories in this condition are scored using only action-token log-probabilities. For a closer comparison with Oracle Beam, a second modified version, referred to as *Context-Only Beam*, is also evaluated. Context-Only Beam adds the model's most probable predicted wall-mask and visible-goal tokens to each candidate trajectory, while the relative goal displacement is updated deterministically from the selected action. It does not add the probabilities of any of these observation tokens to the trajectory score. The predicted tokens can still affect the probabilities of later actions because they form part of the candidate trajectory context. Comparing Context-Only Beam with Oracle Beam therefore investigates the effect of replacing predicted observations with correct simulated observations while retaining the same action-only scoring rule. Improved performance by Oracle Beam would support the hypothesis that observation-prediction errors harm planning, but would not make Oracle Beam a valid planner for the partially observed problem.

Table 5: Observation-ablation results on the 1,196 partially observed fold-0 evaluation queries. Average steps until termination are calculated over all queries.

Method	B	Completion Rate	Avg. Steps to Stop	Wall-Mask Agreement
Beam Search	2	0.8662	8.3838	0.3347
Beam Search	3	0.8487	8.5234	0.3202
Context-Only Beam	2	0.8303	9.0987	0.3163
Context-Only Beam	3	0.8035	9.3821	0.3081
Oracle Beam	2	0.9808	7.1455	1.0000
Oracle Beam	3	0.9858	7.0736	1.0000

Table 5 reports the completion rate, average steps until termination over all queries, and wall-mask agreement for Beam Search, Context-Only Beam, and Oracle Beam. For Beam Search and Context-Only Beam, wall-mask agreement is the proportion of decoded steps at which the first predicted wall mask in the selected candidate trajectory matches the true wall mask at the resulting position after executing the selected action. For Oracle Beam, the wall mask is supplied by the hidden maze simulator rather than predicted by the model, so its agreement is 1 by construction.

Oracle Beam substantially outperformed both Beam Search and Context-Only Beam in terms of completion rate and had lower average steps until termination. Since unsuccessful

queries reach the 19-step limit, this average reflects both the different completion rates and the lengths of the trajectories. It does not by itself show that Oracle Beam found shorter paths among successful queries.

Beam Search outperformed Context-Only Beam in terms of completion rate, average steps until termination, and wall-mask agreement at both beam widths. This suggests that including the probabilities assigned to predicted observation tokens in the trajectory score was beneficial for selecting the next action under the current experimental setup, even though the wall-mask predictions were often incorrect.

Across the three methods, higher wall-mask agreement was associated with higher completion rates. In particular, the strong performance of Oracle Beam supports the hypothesis that inaccurate observation prediction contributes to the lack of improvement from Beam Search. However, this comparison does not by itself establish causation, because Oracle Beam receives privileged true observations and its perfect wall-mask agreement is guaranteed by construction.

Performance changed only slightly between beam widths $B = 2$ and $B = 3$ for all three methods. For Beam Search and Context-Only Beam, this is consistent with the results reported in Table 3. The two beam widths also produced similar Oracle Beam completion rates. Since both rates were already very high, the small difference between them provides limited evidence about the effect of increasing the beam width.

6.3.3 Mixed-Quality Training

The previous experiment suggested that improving the wall mask prediction accuracy will lead to improved performance of wider searches relative to narrower searches. Training a Transformer on a larger dataset containing trajectories of varying degrees of optimality, as described in Section 6.2.1, is expected to improve the prediction accuracy on wall masks, as well as other tokens, partly due to the larger amount of training data and training exposure to wall masks in more trajectory contexts.

Training on trajectories of varying levels of optimality also poses an additional potential advantage: the training trajectories exhibit behaviour that more closely resembles the trajectories that are formed during decoding. Forming a trajectory in an unspecified maze while only having access to local observations and a goal displacement means that the maze needs to be explored, gradually gaining more information on the maze layout as more of the maze is observed, allowing for better-informed moves. It is unrealistic for an agent to immediately consistently choose optimal moves from the starting position, if the initial observations and goal displacements do not provide enough information to determine the complete maze layout and the agent's position therein. Even though the model may eventually learn all the different training maze layouts during training, it still needs to explore an unspecified maze during decoding to some degree before having enough information to determine which of the training mazes it is navigating. If the model is trained only on optimal trajectories, actions associated with exploration, for example recovery behaviour such as moving backwards after a bad move, may be rejected due to the model predicting low probabilities for actions inconsistent with what was found in the training trajectories. Training the model on trajectories that contain suboptimal moves

that were corrected is therefore hypothesised to improve exploration capabilities during decoding.

Table 6 reports the completion rate, average steps until termination over all queries, and wall-mask accuracy for the decoding strategies applied to a model trained on the mixed-quality training dataset.

Table 6: Results on the partially observed fold-0 evaluation queries after training on the mixed-quality dataset. Average steps until termination are calculated over all queries.

Method	Completion Rate	Avg. Steps to Stop	Wall-Mask Accuracy
Greedy Decoding	0.9038	8.1028	–
Beam Search ($B = 2$)	0.9448	7.5543	0.8283
Beam Search ($B = 3$)	0.9523	7.5008	0.8318

Wall-mask accuracy is not reported for Greedy Decoding because it does not predict post-action wall masks during decoding.

Unlike the results from the optimal-trajectory training experiment, wider searches outperformed narrower searches in terms of completion rate. Wider searches also had lower average steps until termination. Because this metric includes unsuccessful queries that reach the step limit, this does not by itself show that wider searches found shorter successful paths.

As in all the previous experiments, actions are selected according to the probabilities assigned by the model, either to individual actions or to candidate token sequences. Since the training data in this experiment contains suboptimal trajectories and unsuccessful trajectories, the model may select suboptimal actions as a consequence of imitating the training trajectories. The training objective does not provide the model with an explicit metric indicating whether one action is better than another in a given situation; it simply aims to imitate behaviour found in the training data. This suggests that a lower average level of optimality among the training trajectories could lead to worse decoding performance, even if the model's prediction accuracy increases. Despite this potential disadvantage, the completion rates of all the decoding strategies improved in this experiment relative to the optimal-trajectory training experiment.

Wall-mask accuracy improved substantially as expected. Averaged across the two beam widths, the wall- mask accuracy is now 0.8300, whereas it was 0.3275 in the optimal trajectory training experiment. The results, however, do not prove that higher wall-mask accuracies lead to greater advantages of wider searches relative to narrower searches, since multiple variables were changed from the previous experiment, and wall-mask prediction accuracy changed alongside them rather than being independently controlled. In particular, both the amount and the nature of the training data were changed. The primary goal of the experiment remains to compare decoding strategies in a partially observed maze problem, where the model was trained on multiple trajectories of varying quality for each start-goal pair, not to investigate the effects of training data or wall-mask prediction accuracy on trajectory-generation performance. Nonetheless, the observed changes between training data, wall-mask accuracy, and overall decoding performance remain useful in understanding what factors should be considered in choosing a decoding algorithm for similar problems.

6.3.4 Rule-Guided Decoding

Table 7 reports the completion rate and average steps until termination for greedy decoding and beam search with local bonus strengths $\lambda \in \{0, 0.25, 0.5, 1, 2\}$. The same model trained on the mixed-quality dataset and the same fold-0 evaluation queries are used for each setting. The $\lambda = 0$ rows correspond to decoding without the local bonus. Average steps until termination are calculated over all queries.

Table 7: Rule-guided decoding results on the partially observed fold-0 evaluation queries. Average steps until termination are calculated over all queries.

λ	Method	Completion Rate	Avg. Steps to Stop
0	Greedy Decoding	0.9038	8.1028
	Beam Search ($B = 2$)	0.9448	7.5543
	Beam Search ($B = 3$)	0.9523	7.5008
0.25	Greedy Decoding	0.8871	8.1839
	Beam Search ($B = 2$)	0.9365	7.5736
	Beam Search ($B = 3$)	0.9423	7.5017
0.5	Greedy Decoding	0.8687	8.3294
	Beam Search ($B = 2$)	0.9247	7.6697
	Beam Search ($B = 3$)	0.9348	7.5485
1	Greedy Decoding	0.8294	8.6572
	Beam Search ($B = 2$)	0.8963	7.9038
	Beam Search ($B = 3$)	0.9156	7.7508
2	Greedy Decoding	0.7617	9.0970
	Beam Search ($B = 2$)	0.8370	8.4147
	Beam Search ($B = 3$)	0.8671	8.3972

The results show a decline in completion rate for all three decoding strategies as λ increases. In this experiment, the local bonus therefore did not improve goal-reaching success over decoding based on the model's predicted probabilities alone. Increasing λ gives the local rule greater influence relative to the model-assigned log-probabilities, which is consistent with the observed decline in completion rate.

Although the particular rule used in this experiment did not improve completion rate at the tested bonus strengths, a different rule or modification to the decoding objective might do so. In the earlier Grid-Bug2 comparison, the learned decoding methods outperformed the deterministic baseline on the more complex mazes. Together with the decline in completion rate when this local rule was added, that result provides further evidence that the trained model supplies useful guidance in this planning setting.

7 Conclusion and Future Work

This work investigated the effect of decoding strategy on planning performance when maze planning is formulated as autoregressive sequence modelling under a learned generative model. In particular, greedy decoding and beam search were compared in terms of goal-reaching success, steps until termination, and computational cost in a fully- and partially observed discrete 2D maze setting.

The results demonstrate that in a fully observed maze environment, beam search is a more effective decoding strategy than greedy decoding for the maze planning problem under the learned trajectory model, both in terms of goal-reaching success and average steps until termination. Beam width $B = 3$ achieved the highest completion rate and the lowest average steps until termination across the full evaluation.

In the partially observed setting considered here, however, wider searches do not necessarily outperform narrower searches in terms of completion rate and average steps until termination. The results suggest that if the trained model has poor prediction accuracy on components of the observation, wider searches provide little advantage over greedy decoding in terms of completion rate and average steps until termination. Conversely, the results suggest that wider searches can outperform narrower searches in terms of completion rate when more accurate information about future observations is available during search. However, a causal relationship between observation-prediction accuracy and the advantage provided by wider searches was not fully established.

In both the partially and fully observed settings investigated, wider searches required more computation time per step. In the fully observed experiment, the reduction in the number of executed steps was sufficient to produce a lower average total compute time per query. In the partially observed experiment, however, wider searches also had longer average total compute times per query because they did not substantially reduce the number of executed steps.

In this work, planning was performed on the same maze layouts that were used for training. Future work could investigate the generalisation of the trained model to unseen maze layouts in both fully and partially observed settings. Such a model would need to learn general maze-navigation strategies and would have greater utility as a maze planner, rather than only generating reasonable trajectories in a small set of mazes.

This objective would pose some challenges. In the fully observed setting, the model currently encodes a maze layout as a single integer, without explicitly representing its structure. The assumption is that the model learns to associate this integer with the corresponding maze layout during training. An arbitrary identifier for an unseen layout would not convey its structure, so the model would require a different encoding of the maze layout on which to condition.

The planner from the partially observed setting could, in principle, already be applied to unseen mazes, since it is conditioned on local observations and a relative goal displacement rather than a maze-layout identifier. However, the model was trained on a small set of mazes with limited data, so whether it has learned general maze-navigation strategies and would perform reasonably well on unseen mazes remains to be investigated.

The current Transformers are not trained to predict rewards associated with actions. Using a dataset that includes rewards, or augmenting the current datasets by adding rewards to the existing trajectories, would allow a model to be trained to predict them. During planning, the decoder could then use predicted rewards to select actions expected to yield higher future rewards, rather than only selecting actions that are more likely to occur in the training trajectories. This could be particularly useful if the training dataset contains suboptimal trajectories, as predicted rewards could help distinguish likely actions from actions expected to have higher future rewards.

If actions can be chosen to maximise expected rewards during decoding, an autoregressive sequence modelling approach could be applied to a range of offline reinforcement learning problems. Benchmark datasets are available for offline reinforcement learning, and the planner could be evaluated on these datasets using different decoding strategies. This would allow its performance and that of the decoding strategies to be compared with other offline reinforcement learning methods.

Furthermore, additional decoding strategies can be investigated. Possible additional strategies include temperature sampling, where the model's logits are scaled by a temperature parameter before probabilities are calculated and sampling takes place, and top-k or nucleus sampling, where sampling is restricted to the k most probable actions or the smallest set of actions whose cumulative probability reaches a threshold p, respectively [6]. These strategies can provide a balance between exploration and exploitation during planning. Modern LLMs commonly combine temperature scaling with nucleus sampling: temperature is applied to the model's logits, the resulting logits are converted to probabilities, and the next token is sampled from the set whose cumulative probability reaches p [7]. It would be interesting to investigate whether this combined approach is more advantageous than greedy decoding or beam search in various planning problems.

In reality, planning problems vary considerably, and autoregressive sequence modelling is only one of many ways to approach them. Its advantages include the ability to use the scalable Transformer architecture and to condition the model on a prefix of past states or observations and actions, allowing it to use information from earlier steps. This work demonstrates that, when autoregressive sequence modelling is used for planning, the choice of decoding algorithm merits attention. The decoding algorithm can affect both trajectory quality and generation speed, so comparing different algorithms can help in designing a more effective planner, especially in practical applications where both trajectory quality and generation speed are critical.

References

[1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[2] Ronald Parr and Stuart Russell. Approximating Optimal Policies for Partially Observable Stochastic Domains. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1088–1094, 1995.

[3] Michael Janner, Qiyang Li, and Sergey Levine. Offline Reinforcement Learning as One Big Sequence Modeling Problem. *Advances in Neural Information Processing Systems*, 2021.

[4] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to Sequence Learning with Neural Networks. *Advances in Neural Information Processing Systems*, 27, 2014.

[5] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc Le. Massive Exploration of Neural Machine Translation Architectures. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 1442–1451, 2017.

[6] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations*, 2020.

[7] Hugging Face. *Transformers: Generation*. Online documentation, accessed 19 October 2026. https://huggingface.co/docs/transformers/main_classes/text_generation.

[8] Matthew Hausknecht and Peter Stone. Deep Recurrent Q-Learning for Partially Observable MDPs. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.

[9] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition. Online manuscript released January 6, 2026. https://web.stanford.edu/~jurafsky/slp3/ed3book_jan26.pdf

[10] Ian Goodfellow, Yoshua Bengio and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[11] Diaeddin Alarnaouti, George Baryannis, Mauro Vallati. *Reformulation techniques for automated planning: a systematic review. The Knowledge Engineering Review*, 38, 2023.

[12] Farama Foundation. *D4RL repository, `d4rl/pointmaze/maze_model.py`*. https://github.com/Farama-Foundation/D4RL/blob/master/d4rl/pointmaze/maze_model.py.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. Advances in Neural Information Processing Systems, 30, 2017.

A Project Plans

This appendix includes both the original project plan and the revised project plan submitted for the project.

Table 8: Original Project Plan.

Date	**Planned Progress**
7 April	Implementation of a minimal end-to-end experimental pipeline. This includes a simple 2D gridworld environment, generation of shortest paths using A*, training of a basic autoregressive trajectory model on the generated paths, and implementation of at least one decoding strategy to produce trajectories from the learned model. The environment will be kept deliberately simple to validate the modelling and decoding pipeline while avoiding triviality.
4 May	Development of a more robust gridworld environment with configurable obstacles, start and goal states, and automated generation of shortest-path trajectories using A*. The environment should support systematic experimentation and dataset generation.
19 May	Implementation of two decoding strategies (e.g., greedy decoding and stochastic sampling) and integration with the autoregressive trajectory model. Initial experiments verifying that trajectories can be generated reliably from the trained model.
26 May	Preliminary evaluation of decoding strategies in the experimental environment. Initial analysis of trajectory quality and success rate. Identification of limitations of the current environment and model.
2 June	Integration of the Maze2D dataset from the D4RL benchmark and adaptation of the modelling pipeline to support training and evaluation on this dataset. Verification that the autoregressive model and decoding procedures operate correctly on the benchmark data.
10 June	Implementation refinements and additional experiments comparing decoding strategies. Collection of experimental results and figures to be included in the interim report.
16 June	Drafting of the interim report, including description of the experimental setup, modelling approach, decoding strategies, and preliminary results.
20 June	Revision and refinement of the interim report. Preparation of figures, tables, and final formatting.
24 June	Final proofreading and preparation for submission of the interim report.
20 July	Submission of the interim report.
20–24 July	Interim presentations.
8 August	Implementation of improvements to the autoregressive trajectory model to better capture trajectory structure in the dataset. Initial experiments validating the training procedure and model behaviour.

Date	Planned Progress
20 August	Implementation and integration of additional decoding strategies, allowing systematic comparison with previously implemented decoding methods.
30 August	Execution of the main experimental evaluation. This will involve systematic comparison of decoding strategies across multiple parameter settings, analysing metrics such as trajectory success rate, path optimality, diversity, and computational cost.
5 September	Analysis of experimental results and preparation of figures and tables summarising the performance of the different decoding strategies.
10 September	Writing and revision of the first full draft of the report describing the modelling approach, decoding strategies, experimental setup, and results obtained so far.
20 October	Implementation of an application demonstrating the practical use of the proposed planning framework. This may involve applying the trained model and decoding procedures to a more complex navigation or planning scenario. Completion and refinement of the final written report will also occur during this stage.
30 October	Submission of final report.
6 November	Submission of the project summary document.
16–20 November	Final project presentation.

Table 9: Revised Project Plan.

Date	Planned Progress
7 April	Construction of a minimal end-to-end Maze2D analysis pipeline. This includes inspection of the D4RL / Maze2D environment structure, loading and visualisation of trajectories, and initial verification of how observations, `qpos`, `qvel`, actions, and goals are represented in the benchmark data.
4 May	Detailed investigation of the geometric relationship between stored Maze2D positions and maze wall layouts. Development of continuous and discrete trajectory visualisations, together with validation procedures to ensure that the plotted trajectories are consistent with the benchmark maze geometry.
19 May	Construction of a discrete trajectory dataset from Maze2D-style mazes. This includes representing trajectories as alternating state and action tokens, discretising maze layouts into grid-based paths, and preparing data in a form suitable for autoregressive sequence modelling.

Date	Planned Progress
26 May	Implementation of greedy decoding and beam search for the discrete planning setting, together with initial experiments verifying that a trained autoregressive model can generate valid trajectories from start-goal queries.
2 June	Expansion from a single Maze2D example to multiple D4RL maze layouts. Construction of a benchmark dataset containing shortest valid trajectories for all connected start-goal pairs across the selected maze family.
10 June	Implementation refinements to the multi-maze pipeline, including tokenisation, trajectory encoding, training-set construction, and preparation of transformer-ready inputs, labels, token-type information, and masks.
16 June	Training of a small causal Transformer on the discrete trajectory data and execution of initial decoding experiments comparing greedy decoding with beam search under multiple maze layouts.
20 June	Aggregation and analysis of preliminary results, including completion rate, average steps to goal, and computational-cost metrics. Preparation of figures, tables, and explanations for inclusion in the interim report.
24 June	Revision and refinement of the interim report, including clarification of the probabilistic modelling formulation, dataset construction, Transformer training procedure, and decoding methodology.
20 July	Submission of the interim report.
20–24 July	Interim presentations.
8 August	Investigation of how decoding performance changes when the trajectory model is trained on sub-optimal trajectories rather than shortest paths, testing the dependence of the current imitation-learning formulation on optimal training data.
20 August	Extension of the planning framework toward an offline reinforcement learning setting, where decoding is guided by predicted reward or return rather than model probability alone.
30 August	Implementation and evaluation of additional decoding strategies, such as stochastic decoding or nucleus sampling, for comparison with greedy decoding and beam search.
5 September	Analysis of the expanded experimental results and preparation of additional figures and tables comparing decoding strategies under the revised settings.
10 September	Writing and revision of the first full draft of the final report, consolidating the modelling formulation, experimental methodology, decoding evaluation, and discussion of results.
20 October	Final refinement of the report and completion of any remaining experimental or presentation material needed for the project submission stage.

Date	Planned Progress
30 October	Submission of final report.
6 November	Submission of the project summary document.
16–20 November	Final project presentation.

B Per-Maze Decoding Results

This appendix reports decoding performance for each maze layout.

Table 10: Decoding results for the OPEN maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.829167	8.041667	0.016394	0.002006
Beam 2	0.970833	4.429167	0.008924	0.001971
Beam 3	1.000000	3.375000	0.007130	0.002033

Table 11: Decoding results for the U_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.733333	8.011111	0.014807	0.001859
Beam 2	0.977778	4.055556	0.007882	0.002033
Beam 3	1.000000	3.688889	0.007291	0.001993

Table 12: Decoding results for the U_MAZE_EVAL maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.866667	6.322222	0.011050	0.001743
Beam 2	1.000000	3.666667	0.007046	0.001833
Beam 3	1.000000	3.666667	0.007423	0.001867

Table 13: Decoding results for the SMALL_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	1.000000	2.250000	0.004439	0.001950
Beam 2	1.000000	1.666667	0.004274	0.002933
Beam 3	1.000000	1.666667	0.007515	0.006156

Table 14: Decoding results for the MEDIUM_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.749231	9.618462	0.017360	0.001794
Beam 2	0.944615	6.163077	0.011570	0.001889
Beam 3	0.987692	5.324615	0.010345	0.001954

Table 15: Decoding results for the MEDIUM_MAZE_EVAL maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.629231	10.852308	0.017826	0.001627
Beam 2	0.907692	6.581538	0.011681	0.001734
Beam 3	0.972308	5.612308	0.010373	0.001783

Table 16: Decoding results for the LARGE_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.665700	12.077778	0.020590	0.001692
Beam 2	0.921256	9.069082	0.017019	0.001850
Beam 3	0.984541	8.198551	0.016133	0.001904

Table 17: Decoding results for the LARGE_MAZE_EVAL maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.690102	11.722942	0.018061	0.001525
Beam 2	0.899167	9.003700	0.015226	0.001658
Beam 3	0.969935	8.001388	0.014145	0.001731

C Partially Observed Per-Maze Results

This appendix reports the complete fold-0 results for the partially observed experiment. Beam 2 and Beam 3 use the token-level beam-search implementation described in the main text.

Table 18: Partially observed decoding results for the OPEN maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Grid-Bug2	1.000000	2.625000	0.000069	0.000029
Greedy	1.000000	2.666667	0.007918	0.003244
Beam 2	1.000000	2.625000	0.075769	0.025944
Beam 3	1.000000	2.666667	0.088987	0.029421

Table 19: Partially observed decoding results for the U_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Grid-Bug2	1.000000	4.666667	0.000102	0.000024
Greedy	0.944444	4.833333	0.014477	0.003329
Beam 2	0.944444	4.944444	0.227514	0.031963
Beam 3	0.944444	4.944444	0.276001	0.035203

Table 20: Partially observed decoding results for the U_MAZE_EVAL maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Grid-Bug2	1.000000	5.555556	0.000109	0.000023
Greedy	1.000000	4.333333	0.011755	0.002865
Beam 2	0.888889	4.833333	0.224188	0.029287
Beam 3	0.888889	4.833333	0.276476	0.034410

Table 21: Partially observed decoding results for the SMALL_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Grid-Bug2	1.000000	1.600000	0.000042	0.000028
Greedy	1.000000	1.600000	0.004541	0.003082
Beam 2	1.000000	1.600000	0.024918	0.013684
Beam 3	1.000000	1.600000	0.031388	0.016532

Table 22: Partially observed decoding results for the MEDIUM_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Grid-Bug2	0.630769	11.153846	0.000201	0.000020
Greedy	0.930769	6.646154	0.016204	0.002583
Beam 2	0.946154	6.192308	0.284485	0.037189
Beam 3	0.938462	6.238462	0.346789	0.043624

Table 23: Partially observed decoding results for the MEDIUM_MAZE_EVAL maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Grid-Bug2	0.653846	10.723077	0.000194	0.000021
Greedy	0.853846	6.969231	0.018254	0.002739
Beam 2	0.853846	6.992308	0.366446	0.038627
Beam 3	0.846154	7.000000	0.448500	0.046109

Table 24: Partially observed decoding results for the LARGE_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Grid-Bug2	0.345411	15.340580	0.000277	0.000019
Greedy	0.867150	9.388889	0.023774	0.002598
Beam 2	0.833333	9.635266	0.532678	0.046158
Beam 3	0.804348	9.896135	0.673938	0.056137

Table 25: Partially observed decoding results for the LARGE_MAZE_EVAL maze.

Strategy	Completion Rate	Avg. Steps to Stop	Avg. Compute Time (s)	Avg. Time per Step (s)
Grid-Bug2	0.339492	14.752887	0.000270	0.000020
Greedy	0.845266	9.508083	0.028677	0.002971
Beam 2	0.856813	9.270208	0.593554	0.052472
Beam 3	0.840647	9.385681	0.717095	0.062262